

PLP - Primer Parcial - 1^{er} cuatrimestre de 2025 - Parte 1

Corrigió Damásin

#Orden	Libreta	Apellido y Nombre	Nota
25	1110123	Bailleres Laura	✓

Esta parte del examen se aprueba respondiendo correctamente al menos el 75% de las preguntas. Es necesario aprobar esta parte para acceder a la corrección de la segunda parte. Responder las preguntas en los espacios indicados.

Pregunta 1: La siguiente definición ¿utiliza recursión explícita? Justificar.

```
esHoja :: AB a -> Bool
esHoja Nil = False
esHoja (Bin i r d) = esNil i && esNil d
  where esNil Nil = True
        esNil _ = False
```

Respuesta: no porque no es una función recursiva y es Nil tampoco ^{esHoja} ✓

Pregunta 2: Indicar el tipo del esquema de recursión estructural foldABNV para el siguiente tipo:

```
data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a)
```

Respuesta: foldABNV :: (a -> b) -> (a -> b -> b) -> (b -> a -> b -> b) -> ABNV a -> b ✓

Pregunta 3: ¿Qué tipo de recursión utiliza la siguiente definición? (Indicar la más específica).

```
listasQueSuman :: Int -> [[Int]]
listasQueSuman 0 = [[]]
listasQueSuman n | n > 0 = [x : xs | x <- [1..n], xs <- listasQueSuman (n-x)]
```

- a. Estructural
- b. Primitiva
- c. Global
- d. Iterativa/a la cola

Respuesta: c) ✓

Pregunta 4: Indicar cuál de las siguientes divisiones en casos puede justificarse por un Lema de Generación.

a. Hay 2 casos:

- altura i ≥ altura d
- altura i < altura d

(Con i, d :: AB a).

b. La lista xs puede ser:

- []
- [x]
- x:y:ys

(Con x, y :: a, ys :: [a]).

c. El árbol binario t puede ser:

- Nil
- Bin i r d

(Con r :: a, i y d :: AB a).

Respuesta: c) ✓

Pregunta 5: Definir el predicado unario P que se podría usar para demostrar la siguiente propiedad por inducción estructural:

$\forall xs :: [a]. \forall ys :: [a]. \text{length} (\text{entrelazar } xs \text{ } ys) = \text{length } xs + \text{length } ys$

Respuesta: P(xs) = $\forall ys :: [a]. \text{length} (\text{entrelazar } xs \text{ } ys) = \text{length } xs + \text{length } ys$ ✓

Pregunta 6: Si se quisiera demostrar que vale $\neg P \vee Q \vdash Q \vee (P \Rightarrow Q)$ por deducción natural, ¿cuál sería el error al aplicar el siguiente paso?

$$\frac{\neg P \vee Q \vdash Q}{\neg P \vee Q \vdash Q \vee (P \Rightarrow Q)} (V_{i1})$$

Respuesta: que $(Q \vee (P \Rightarrow Q))$ puede ser verdadero con Q falso, y mi premisa ~~$\neg P \vee Q \vdash Q$~~ puede ser falsa.

Pregunta 7: Indicar a qué término reduce en un paso la siguiente expresión, y mediante qué regla(s):
if isZero(zero) then Pred(Succ(zero)) else zero

Respuesta: ^{E: isZero} $\rightarrow$ if true then Pred(Succ(zero)) else zero También usa la regla E-If

Pregunta 8: Sea la extensión del Cálculo Lambda con listas vista en la guía:

$$M ::= \dots \mid []_r \mid M :: M \mid \text{case } M \text{ of } \{ [] \rightsquigarrow M \mid h :: t \rightsquigarrow M \} \mid \text{foldr } M \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow M$$

Indicar el error en la siguiente regla de cómputo para foldr:

$$\frac{\text{foldr } V_1 :: V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \rightarrow N}{\text{foldr } V_1 :: V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \rightarrow N} \text{ (E-FOLDR-CONS)}$$

Respuesta: tengo que usar h y r en N . ya que pueden ocurrir en N como variables libres.

Corrigió *Damián*

PLP - Primer Parcial - 1^{er} cuatrimestre de 2025 - Parte 2

#Orden	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
25	1110/23	Bailleres Laura	MB	MB	B	P

Las notas para cada ejercicio son: -, I, R, B, MB, E. Este examen se aprueba obteniendo al menos dos ejercicios B y uno R, y se promociona con al menos dos ejercicios MB y uno B. Es posible obtener una aprobación condicional con un ejercicio MB, uno B y uno I. Esto requiere entregar algo que contribuya a la solución del ejercicio. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

Se define el tipo de datos ABNV a , que representa árboles binarios no vacíos con elementos de tipo a :

$\text{data ABNV } a = \text{Hoja } a \mid \text{Uni } a \ (\text{ABNV } a) \mid \text{Bi } (\text{ABNV } a) \ a \ (\text{ABNV } a)$

Definimos el siguiente árbol para los ejemplos:

$\text{abnv} = \text{Bi } (\text{Uni } 2 \ (\text{Hoja } 1)) \ 3 \ (\text{Bi } (\text{Hoja } 4) \ 5 \ (\text{Uni } 2 \ (\text{Hoja } 7)))$

- Definir las funciones foldABNV y recABNV , que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo ABNV a . Solo en este inciso se permite usar recursión explícita.
- Definir la función $\text{elemABNV} :: \text{Eq } a \Rightarrow a \rightarrow \text{ABNV } a \rightarrow \text{Bool}$, que indica si un elemento pertenece a un árbol.

Por ejemplo: $\text{elemABNV } 7 \ \text{abnv} \rightsquigarrow \text{True}$.

- Definir la función $\text{reemplazarUno} :: \text{Eq } a \Rightarrow a \rightarrow a \rightarrow \text{ABNV } a \rightarrow \text{ABNV } a$ que, dados dos elementos x e y y un árbol, devuelve un árbol como el original, pero reemplazando la primera aparición de x por y (la primera desde la raíz yendo de izquierda a derecha, en el orden de `preorder`).

Por ejemplo:

$\text{reemplazarUno } 2 \ 5 \ \text{abnv} \rightsquigarrow \text{Bi } (\text{Uni } 5 \ (\text{Hoja } 1)) \ 3 \ (\text{Bi } (\text{Hoja } 4) \ 5 \ (\text{Uni } 2 \ (\text{Hoja } 7)))$.
 $\text{reemplazarUno } 2 \ 5 \ (\text{Hoja } 1) \rightsquigarrow \text{Hoja } 1$.

- Definir la función $\text{nivel} :: \text{ABNV } a \rightarrow \text{Int} \rightarrow [a]$, que liste todos los elementos del nivel indicado, de izquierda a derecha, siendo 0 el nivel de la raíz. Si el nivel no existe (ya sea por ser negativo o por superar la altura del árbol), devolver $[]$.

Por ejemplo:

$\text{nivel } \text{abnv} \ 1 \rightsquigarrow [2, 5]$.
 $\text{nivel } \text{abnv} \ 2 \rightsquigarrow [1, 4, 2]$.
 $\text{nivel } \text{abnv} \ 4 \rightsquigarrow []$.

Pista: aprovechar la curriificación y utilizar evaluación parcial.

Ejercicio 2 - Demostración de propiedades

- Considerar las siguientes definiciones:

$\text{data AB } a = \text{Nil} \mid \text{Bin } (\text{AB } a) \ a \ (\text{AB } a)$

$\text{elemAB} :: \text{Eq } a \Rightarrow a \rightarrow \text{AB } a \rightarrow \text{Bool}$

$\{P0\} \ \text{elemAB} \ _ \ \text{Nil} = \text{False}$

$\{P1\} \ \text{elemAB } x \ (\text{Bin } i \ r \ d) = (x == r) \mid \mid (\text{elemAB } x \ i) \mid \mid (\text{elemAB } x \ d)$

$\text{mapAB} :: (a \rightarrow b) \rightarrow \text{AB } a \rightarrow \text{AB } b$

$\{M0\} \ \text{mapAB} \ _ \ \text{Nil} = \text{Nil}$

$\{M1\} \ \text{mapAB } f \ (\text{Bin } i \ r \ d) = \text{Bin } (\text{mapAB } f \ i) \ (f \ r) \ (\text{mapAB } f \ d)$

Sean dos tipos a y b tales que valen $\text{Eq } a$, $\text{Eq } b$, y por ende también la siguiente propiedad:

$$\{\text{CONGRUENCIA } ==\} \forall x::a. \forall y::a. \forall f::a \rightarrow b. x == y \Rightarrow f x == f y$$

Demostrar la siguiente propiedad:

$$\forall t::AB \ a. \forall x::a. \forall f::a \rightarrow b. (\text{elemAB } x \ t \Rightarrow \text{elemAB } (f \ x) \ (\text{mapAB } f \ t))$$

Se permite definir macros (i.e., poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio escribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes. Recordar también que los $=$ de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos.

Sugerencia: tener en cuenta que $\forall x::\text{Bool}. \forall y::\text{Bool}. \forall z::\text{Bool}. (x \Rightarrow y) \Rightarrow (x \Rightarrow z \vee y)$.

b) Demostrar el siguiente teorema usando Deducción Natural, sin utilizar principios clásicos:

$$\rho \Rightarrow (\sigma \vee (\rho \Rightarrow \tau)) \Rightarrow (\sigma \vee \tau)$$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda simplemente tipado para modelar **listas ordenadas**. Una lista ordenada se construye de manera similar a una lista común, pero sus observadores se comportan de manera diferente: la cabeza de la lista es siempre el mínimo elemento, y su cola es el resto de los elementos.

Estas listas tienen como precondition que el tipo de sus elementos cuente con una operación de comparación $<$ (no es necesario escribir nada al respecto, suponemos que vale).

Se extienden los tipos y expresiones de la siguiente manera:

$$\begin{aligned} \tau &::= \dots \mid [\tau]^< \\ M &::= \dots \mid []_\tau^< \mid M ::< M \mid \text{head}^<(M) \mid \text{tail}^<(M) \end{aligned}$$

donde:

- $[\tau]^<$ es el tipo de las listas ordenadas con elementos de tipo τ .
- $[]_\tau^<$ es una lista ordenada vacía que admite valores de tipo τ .
- $M_1 ::< M_2$ es la lista resultante de agregar el elemento M_1 a la lista M_2^1 .
- $\text{head}^<(M)$ denota el mínimo elemento de la lista M .
- $\text{tail}^<(M)$ es la lista ordenada con todos los elementos de M excepto el mínimo (si M tiene más de una aparición de su mínimo elemento, se elimina una de ellas).

a) Introducir las reglas de tipado para la extensión propuesta.

b) Definir el conjunto de valores y las nuevas reglas de reducción en un paso. Tener en cuenta que una lista vacía no tiene cabeza ni cola.

Pista: puede ser necesario mirar más de un nivel de un término para saber a qué reduce.

c) Mostrar paso por paso cómo reduce la expresión:

$$\text{head}^<((\lambda x: \text{Nat}. 2) \text{zero} ::< 1 ::< []_{\text{Nat}}^<)$$

Se pueden considerar dadas las siguientes reglas:

$$\frac{M_1 \rightarrow M'}{M_1 < M_2 \rightarrow M' < M_2} (\text{E-}<_1) \quad \frac{M_2 \rightarrow M'}{V < M_2 \rightarrow V < M'} (\text{E-}<_2)$$

$$\frac{}{V < \text{zero} \rightarrow \text{False}} (\text{E-}<_F) \quad \frac{}{\text{zero} < \text{Succ}(V) \rightarrow \text{True}} (\text{E-}<_T)$$

$$\frac{}{\text{Succ}(V_1) < \text{Succ}(V_2) \rightarrow V_1 < V_2} (\text{E-}<_S)$$

¹Notar que M_1 no necesariamente es la cabeza de la lista, y que $::<$ es un constructor, no es responsable de ordenar los elementos en la lista. El orden aparece al observarla. Por ejemplo, $1 ::< 2 ::< []_{\text{Nat}}^<$ y $2 ::< 1 ::< []_{\text{Nat}}^<$ son dos valores de tipo $[\text{Nat}]^<$ que tienen la misma cabeza y la misma cola.

EJERCICIO 1 : PROGRAMACIÓN FUNCIONAL

data ABNV a = Hoja a | Uni a (ABNV a)
| Bi (ABNV a) a (ABNV a)

a) foldABNV :: (a → b) → (a → b → b) →
(b → a → b → b) → ABNV a → b

foldABNV CHOja CUni CBi t =

case t of

Hoja x → CHOja x

Uni n u → CUni n (rec u)

Bi b1 m b2 → CBi (rec b1) m (rec b2)

where rec = foldABNV CHOja CUni CBi

recABNV :: (a → b) → (a → ABNV a → b → b)

(ABNV a → b → a → ABNV a → b → b) → ABNV a → b

recABNV CHOja CUni CBi t

case t of

Hoja x → CHOja x

~~Uni n abv m → CUni n abv (rec u)~~

~~Bi ab1 b2 m ab2 b3 →~~

~~CBi ab1 (rec b1) m ab2 (rec b2)~~

~~where rec = recABNV CHOja CUni CBi~~

Uni n u → CUni n u (rec u)

Bi b1 m b2 → CBi b1 (rec b1) m b2 (rec b2)

$$\text{tail}^<(V_1 :: []^<) \rightarrow []^< \pi \quad \text{tail}_2 \checkmark$$

$$\text{tail}^<(V_1 :: V_2 :: V_3) \rightarrow \text{if } (V_1 < V_2) \text{ then } V_2 :: (\text{tail}^<(V_2 :: V_3)) \text{ else } \dots$$

X solo se podía usar "tail 2"

$$\text{tail}^<(V_1 :: V_2 :: V_3) \rightarrow \text{if } (V_1 == \text{head}^<(V_2 :: V_3)) \text{ then } V_2 :: V_3 \checkmark \text{ else } V_1 :: (\text{tail}^<(V_2 :: V_3)) \checkmark$$

La guarda tiene que ser $V_1 < \text{head}^<(V_2 :: V_3)$

Faltan reglas de congruencia

c) $\text{head}^<((\lambda x: \text{Nat}. _) \text{zero} :: []^<_{\text{Nat}})$

head 2 pide que la lista no sea un valor

$\rightarrow \text{if } ((\lambda x: \text{Nat}. _) \text{zero}) < []^<_{\text{Nat}} \text{ then}$

$\text{head}^<((\lambda x: \text{Nat}. _) \text{zero}) :: []^<_{\text{Nat}} \text{ else}$

$\text{head}^<([]^<_{\text{Nat}})$

$\rightarrow \text{if } (z \leq []^<_{\text{Nat}})$

$\text{then head}^<((\lambda x: \text{Nat}. _) \text{zero}) :: []^<_{\text{Nat}} \text{ else head}^<([]^<_{\text{Nat}})$

$\text{head}^<([]^<_{\text{Nat}})$

Falta un poro

$\rightarrow \text{if False then head}^<((\lambda x: \text{Nat}. _) \text{zero}) :: []^<_{\text{Nat}}$

$\text{else head}^<([]^<_{\text{Nat}})$

$\rightarrow \text{if False then head}^<([]^<_{\text{Nat}})$

$\rightarrow \text{head}^<([]^<_{\text{Nat}})$

$[]^<_{\text{Nat}}$

reescribo

$$\text{succ}(\text{succ}(\text{zero})) < \text{succ}(\text{zero}) \rightarrow \text{succ}(\text{zero}) < \text{zero} \rightarrow \text{False}$$

(E-C) *(E-CF)*

Falta el poro donde aparece este guardo en todo el término

EJERCICIO 2: DEMOSTRACIÓN DE PROPIEDADES

a) data AB a = NU | Bin (AB a) a (AB a)

quiero demostrar la propiedad

$$\forall t :: AB\ a. \forall x :: a. \forall f :: a \rightarrow b.$$

$$(elem\ x\ t \Rightarrow elem\ AB\ (f\ x)\ (map\ AB\ f\ t))$$

defino el predicado unario.

$$P(t) = \forall x :: a. \forall f :: a \rightarrow b. (elem\ x\ t \Rightarrow elem\ AB\ (f\ x)\ (map\ AB\ f\ t))$$

hago inducción en la estructura del árbol.

tengo un constructor base (NU) y un constructor recursivo (Bin (AB a) a (AB a)). si $P(NU)$ y

$P(i) \rightarrow P(d)$ $\forall i :: AB\ a. \forall x :: a$ ~~base~~ ^{inducción} ~~base~~ ^{inducción} vale, entonces $\forall t :: AB\ a. P(t)$.

Caso NU

$$P(NU) = \forall x :: a. \forall f :: a \rightarrow b.$$

$$(elem\ AB\ x\ NU \Rightarrow elem\ AB\ (f\ x)\ (map\ AB\ f\ NU))$$

como quiero demostrar una implicación, primero veo el antecedente y después me fijo qué pasa con el consecuente.

$$elem\ AB\ x\ NU = \{ \text{false} \}$$

False.

como el antecedente de la implicación es falso, el consecuente puede tomar cualquier valor y la implicación siempre ~~es~~ ^{será} verdadera.

Caso $\{Bin \vdash r \ d\}$

$$P(Bin \vdash r \ d) = \forall x :: a. \forall f :: a \rightarrow b.$$

$$(elemAB \ x \ (Bin \vdash r \ d) \Rightarrow elemAB \ (f \ x) \ (mapAB \ f \ (Bin \vdash r \ d)))$$

HIPÓTESIS INDUCTIVAS: $\bigoplus \forall x :: a. \forall f :: a \rightarrow b.$

$$\{h.i. 1\} \quad P(i) \stackrel{\checkmark}{=} \bigoplus (elemAB \ x \ i \Rightarrow elemAB \ (f \ x) \ (mapAB \ f \ i))$$

$$\{h.i. 2\} \quad P(d) \stackrel{\checkmark}{=} \bigoplus (elemAB \ x \ d \Rightarrow elemAB \ (f \ x) \ (mapAB \ f \ d))$$

~~HI (general) TESTS: Ad. $P(i) \wedge P(d)$~~

como quiero demostrar una implicación, primero veo el antecedente y después me fijo qué pasa con el consecuente.

$$elemAB \ x \ (Bin \vdash r \ d) = \{P1\} \checkmark$$

$$(x == r) \parallel (elemAB \ x \ i) \parallel (elemAB \ x \ d) = \{asoc.\} \checkmark$$

$$(x == r) \parallel ((elemAB \ x \ i) \parallel (elemAB \ x \ d))$$

por lema de generación de Bool, $(x == r)$ puede ser True o False. $\checkmark$

(Hay varios paréntesis innecesarios)

Caso True

$$True \parallel ((elemAB \ x \ i) \parallel (elemAB \ x \ d)) = \{Bool\} \checkmark$$

True.

recuerdo que yo quiero demostrar una implicación. como mi antecedente es verdadero, el consecuente también debe serlo. $\checkmark$

$$elemAB \ (f \ x) \ (mapAB \ f \ (Bin \vdash r \ d)) = \{M1\} \checkmark$$

$$elemAB \ (f \ x) \ (Bin \ (mapAB \ f \ i) \ (f \ r) \ (mapAB \ f \ d)) = \{P1\} \checkmark$$

$$((f \ x) == (f \ r)) \parallel elemAB \ (f \ x) \ (mapAB \ f \ i) \parallel elemAB \ (f \ x) \ (mapAB \ f \ d) =$$

como estoy en el caso $(x == r)$, por CONGRUENCIA $= \{ \}$ $\checkmark$

$$True \parallel elemAB \ (f \ x) \ (mapAB \ f \ i) \parallel elemAB \ (f \ x) \ (mapAB \ f \ d) = \{Bool\} \checkmark$$

True.

entonces tengo que $(True \Rightarrow True) = True \checkmark$

Caso Falso

$$\text{False} \Rightarrow ((\text{elemAB } x \ i) \Rightarrow (\text{elemAB } x \ d)) = \{\text{Bool}\}$$

$$(\text{elemAB } x \ i) \Rightarrow (\text{elemAB } x \ d)$$

por lema de generación de Bool, $(\text{elemAB } x \ i) \Rightarrow (\text{elemAB } x \ d)$ puede ser True o False.

Caso Falso lo sea, $(\text{elemAB } x \ i) = \text{False}$ y $(\text{elemAB } x \ d) = \text{False}$
 como el antecedente de la implicación es falso, el consecuente puede tomar cualquier valor y la implicación siempre sería verdadera.

Caso True o $(\text{elemAB } x \ i)$ o $(\text{elemAB } x \ d)$

Falta seguir. Deberías usar es verdadera. asumo $(\text{elemAB } x \ i) = \text{True}$
 lema de gen. sobre elemAB el caso $(\text{elemAB } x \ d)$ es análogo.

o, mejor, usar propiedades de Bool
 para simplificar la demo.

$$\text{True} \Rightarrow (\text{elemAB } x \ d) = \{\text{Bool}\}$$

¿por qué copio True.
 este antecedente?

Podría haber desarrollado esto antes de separar en casos, para no repetir.

como el antecedente de la implicación es verdadero, el antecedente también tiene que serlo.

$$(\text{elemAB } x \ i) \Rightarrow (\text{elemAB } (f \ x) (\text{mapAB } f (\text{Bin } i \ r \ d))) = \{\text{Map}\}$$

$$(\text{elemAB } x \ i) \Rightarrow (\text{elemAB } (f \ x) (\text{Bin } (\text{mapAB } f \ i) (f \ r) (\text{mapAB } f \ d))))$$

$$(\text{elemAB } x \ i) \Rightarrow ((f \ x == f \ r) \Rightarrow \text{elemAB } (\text{map } f \ i) \parallel \text{elemAB } (f \ x) (\text{map } f \ d)))$$

$$(\text{elemAB } x \ i) \Rightarrow (\text{False} \parallel \text{elemAB } (f \ x) (\text{map } f \ i) \parallel \text{elemAB } (f \ x) (\text{map } f \ d)))$$

$$\text{elemAB } (f \ x) (\text{map } f \ d) = \{\text{Bool}\}$$

$$(\text{elemAB } x \ i) \Rightarrow (\text{elemAB } (f \ x) (\text{map } f \ i) \parallel \text{elemAB } (f \ x) (\text{map } f \ d)))$$

de azucar.
 $\{P2\}$
 $\{ \text{cond.} \} = \{ \}$
 $\downarrow$
 esto es en el caso
 $(x \neq r)$
 False

La prop CONGRUENCIA
 no vale si se usa
 Es una implicación

recuerdo que $\{h.i.1\} =$

$$\text{elemAB } x \ i \Rightarrow \text{elemAB } (f x) \ (\text{map AB } f i)$$

y también tengo $\{1\text{ema}\}$

$$\forall x :: \text{Bool}. \forall y :: \text{Bool}. \forall z :: \text{Bool}. (x \Rightarrow y) \Rightarrow (x \Rightarrow z \vee y)$$

entonces por $\{h.i.1\}$ y $\{1\text{ema}\}$,

$$(\text{elemAB } x \ i) \Rightarrow (\text{elemAB } (f x) \ (\text{map AB } f i)) \quad \checkmark$$

$$\text{elemAB } (f x) \ (\text{map } f \ a)) = \text{True} \quad \square$$

b)



$$\frac{Ax}{\Gamma, (p \Rightarrow \pi) \vdash (p \Rightarrow \pi)}$$

Ax

$$\frac{Ax}{\Gamma, (p \Rightarrow \pi) \vdash p}$$



~~proposition~~

Ax

$$\frac{Ax}{\Gamma, \sigma \vdash \sigma}$$

V_{1,2}

$$\frac{V_{1,2}}{\Gamma, (p \Rightarrow \pi) \vdash \pi}$$

V_{1,2}

$$\Gamma \vdash (\sigma \vee (p \Rightarrow \pi))$$

$$\Gamma, \sigma \vdash \sigma \vee \pi$$

$$\Gamma, (p \Rightarrow \pi) \vdash \sigma \vee \pi$$

$$\Gamma = P, (\sigma \vee (p \Rightarrow \pi)) \vdash \sigma \vee \pi$$

$$P \vdash (\sigma \vee (p \Rightarrow \pi)) \Rightarrow (\sigma \vee \pi)$$

$$\vdash P \Rightarrow (\sigma \vee (p \Rightarrow \pi)) \Rightarrow (\sigma \vee \pi)$$

EJERCICIO 3 : CÁLCULO LAMBDA TIPADO

$$\tau ::= \dots \mid [\tau]^<$$

$$M ::= \dots \mid []_{\tau}^< \mid M ::^< M \mid \text{head}^<(M) \mid \text{tail}^<(M)$$

a)

$$\frac{}{\Gamma \vdash []_{\tau}^< : [\tau]^<} \text{T.Vacía} \checkmark$$

$$\frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : [\tau]^<}{\Gamma \vdash M ::^< N : [\tau]^<} \text{T.Agr} \checkmark$$

$$\frac{\Gamma \vdash M : [\tau]^<}{\Gamma \vdash \text{head}^<(M) : \tau} \text{T.head} \checkmark$$

$$\frac{\Gamma \vdash M : [\tau]^<}{\Gamma \vdash \text{tail}^<(M) : [\tau]^<} \text{T.tail} \checkmark$$

b)

$$V ::= \dots \mid []_{\tau}^< \mid V ::^< V \checkmark$$

reglas de ~~comparación~~ reducción

$$\text{head}^<([V_1 ::^< []_{\tau}^<]) \rightarrow V_1 \quad \text{head}_1 \checkmark$$

$$\text{head}^<([V_1 ::^< V_2 ::^< V_3]) \rightarrow \text{if } V_1 < V_2 \text{ then } \text{head}^<([V_1 ::^< V_3]) \text{ else } \text{head}^<([V_2 ::^< V_3]) \quad \text{head}_2 \checkmark$$

b)

elemABNV :: Eq a => a -> ABNV a -> Bool

elemABNV n =

foldABNV (\x -> x == n)

(\x b -> x == n || b)

(\b1 x b2 -> x == n || (b1 || b2))

c)

reemplazarUno :: Eq a => a -> a -> ABNV a -> ABNV a

reemplazarUno e1 e2 =

recABNV

(\x -> if (x == e1) then (Haja e2) else (Haja x))

(\x u b -> if (x == e1) then (uni e2 u) else

(b))

(\ab1 b1 x ab2 b2 ->

if (x == e1) then (Bi ab1 e2 ab2)

else (if (elemABNV b1) then (b1) else (b2))

Si b1 x ab1

Si ab1 x b2

d)

nivel :: ABNV a -> Int -> [a]

nivel = foldABNV

(\x n -> if (n == 0) then [x] else [])

(\x u n -> if (n == 0) then [x] else (u (n-1)))

(\x n -> if (n == 0) then [x] else [])

(\x u n -> if (n == 0) then [x] else (u (n-1)))

(\b1 x b2 n -> if (n == 0) then [x] else (b1 (n-1) ++ b2 (n-1)))

→ como Haskell evalúa de forma perezosa, el orden de la lista de salida va a quedar de izq. a

derecha en el árbol

El orden de la lista no tiene que ver con la forma de evaluación